

Peruvian Food Reviews

Analyze the sentiment of one of the best cuisine

Nome: Franco Eduardo Tejada Toledo

Pais: Peru



O Peru é um dos melhores destinos culinários do mundo. Este país possui diversos climas e andares ecológicos, onde vários cultivos foram desenvolvidos. Dessa forma, conta com uma grande quantidade de insumos naturais e únicos. Assim, a comida peruana é uma culinária de opostos: quente e frio no mesmo prato. Sabores ácidos misturando-se com o amiláceo. Robusta e delicada ao mesmo tempo. Esse equilíbrio ocorre porque a comida tradicional peruana se baseia em especiarias e sabores marcantes, que vão do fresco e limpo ao denso e profundo. Cada sabor contrapõe ou suaviza o outro. Embora muitas pessoas vejam o Peru como uma terra de montanhas cobertas por nuvens e ruínas de civilizações antigas, o verdadeiro tesouro do Peru é a sua rica herança culinária. Ingredientes e técnicas de cozinha da África, Europa e Leste Asiático se reúnem em uma mistura encantadora que é totalmente única no mundo inteiro. Mas que tipo de comida os peruanos comem? E quais restaurantes você deveria visitar?

1. Contexto de Negócios e Perguntas

Este conjunto de dados do Kaggle contém informações coletadas do GooglePlaces e do Tripadvisor usando Selenium, Requests, BeautifulSoup e Rvest. Mais informações sobre a raspagem de dados (web scraping) utilizada estão neste repositório do GitHub.

O conteúdo aqui possui grande parte (no momento não todas) das avaliações de restaurantes em Lima, Peru, entre 2010 e 2021. No total, existem mais de 8.791 restaurantes e mais de 1.160.666 avaliações, com um total de 20 recursos (*features*) com alta diversidade: geoespaciais, texto, data, categóricos e numéricos!

Este conjunto possui duas seções gerais. A primeira é Restaurants. Esta contém informações gerais e geoespaciais. A segunda é Reviews. Esta contém a interação entre o usuário e o restaurante; dessa forma, é possível ver a satisfação do cliente com o serviço.

Existe uma possível terceira seção: Users. Esta informação talvez seja adicionada em dois meses.

Sobre a metodologia de coleta, ela é explicada abaixo:

- A amostra: As avaliações coletadas são as mais recentes de todos os restaurantes possíveis na província de Lima.
- Conjunto de itens: Por um lado, os usuários. Por outro lado, os restaurantes.
- Conjunto de variáveis: Existem duas tabelas gerais. Veja as informações abaixo.

Github: <https://github.com/francotejada/Eng-Dados-Food-Reviews>

1.1. Objetivo

Analisar avaliações de restaurantes peruanos para identificar padrões relacionados à satisfação dos clientes, considerando a nota atribuída, a categoria do estabelecimento e as características das avaliações, por meio de um pipeline ETL implementado no Databricks.

1.2. Perguntas:

Pergunta 1: Quais são os restaurantes mais populares e mais bem avaliados?

Pergunta 2: Quais distritos concentram a maior popularidade e qualidade?

Pergunta 3: Quais restaurantes se destacam em relação à média do seu distrito?

Pergunta 4: Quais emoções caracterizam as opiniões positivas e negativas?

Pergunta 5: Quais restaurantes combinam alta popularidade com alta satisfação?

2. Carga dos Dados

A carga de dados do dataset "Peruvian Food Reviews" no Databricks foi estruturada em duas fases principais: o armazenamento dos arquivos brutos via Unity Catalog (Volumes) e a leitura/ingestão utilizando a API do PySpark.

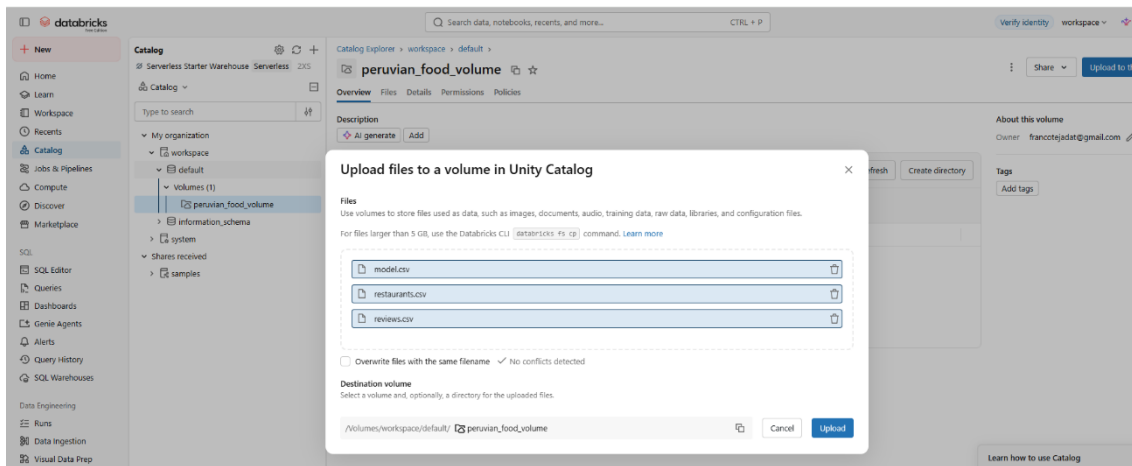
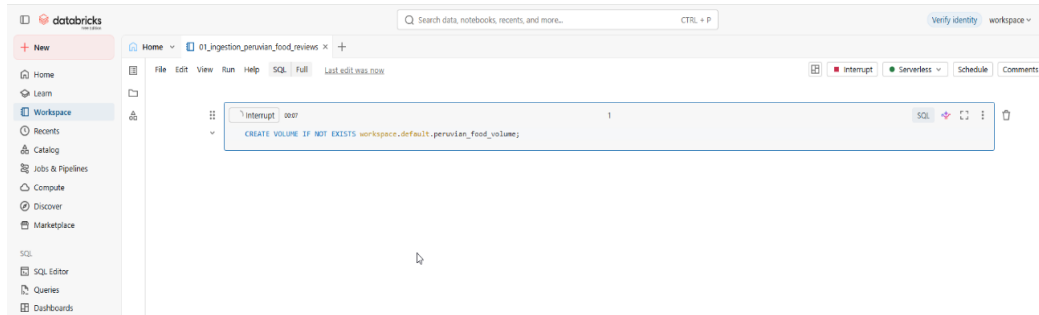
A implementação ocorreu da seguinte forma:

a. Armazenamento no Databricks (Volumes) O processo inicia com a criação de um repositório interno para receber os arquivos baixados do Kaggle. Um volume foi inicializado via SQL:

```
CREATE VOLUME IF NOT EXISTS workspace.default.peruvian_food_volume;
```

Conforme o explorador de catálogos, os três arquivos brutos foram carregados fisicamente para /Volumes/workspace/default/peruvian_food_volume:

- restaurants.csv (1.45 MB)
- reviews.csv (151.81 MB)
- model.csv (28.91 MB)



b. Ingestão e Parametrização via PySpark A leitura dos dados para a memória do cluster (transformação em DataFrames) foi executada no notebook 01_ingestion_peruvian_food_reviews. O código em Python demonstra um cuidado específico com os metadados e o formato dos textos:

Python

```
spark.read.format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .option("multiLine", "true") \
    .option("escape", "'') \
    .load(f'{base_path}/{<nome_do_arquivo>.csv")
```

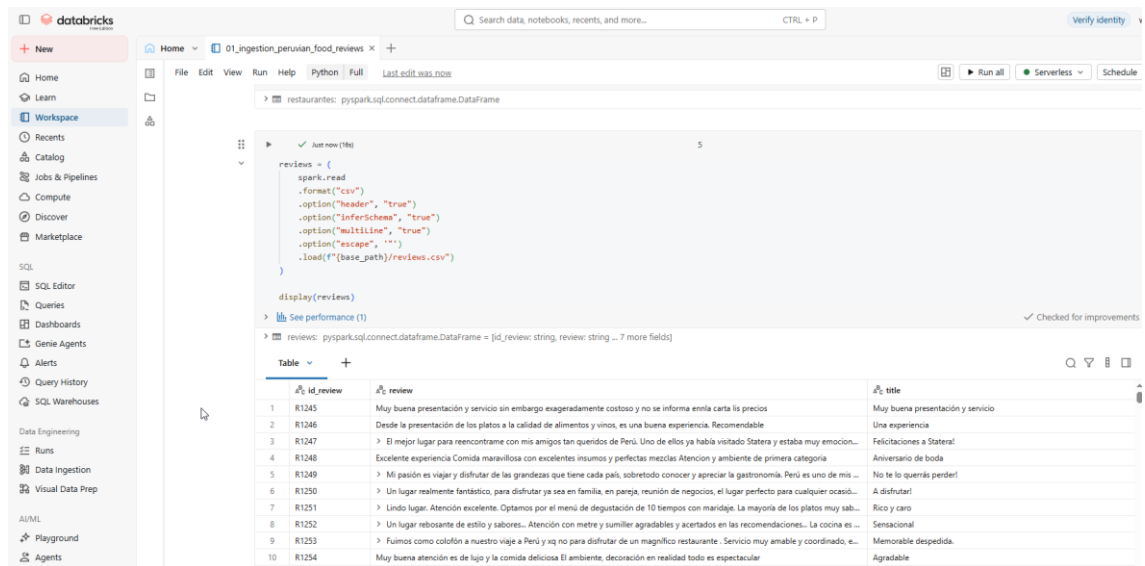
- header e inferSchema: Utilizados para capturar automaticamente os nomes das colunas e tipificar os dados (ex: converter as notas de sentimentos no arquivo *model* para double).

- multiLine e escape: Parâmetros críticos para datasets de processamento de linguagem natural (NLP). Eles garantem que avaliações (reviews) que contenham quebras de linha (\n) ou aspas no meio do texto não desestruturem o arquivo CSV durante a leitura.

Validação Volumétrica da Carga: A etapa foi concluída com um teste de contagem de linhas e colunas para atestar a integridade dos dados extraídos:

- RESTAURANTES: 9.871 linhas | 13 colunas
- REVIEWS: 1.345.723 linhas | 9 colunas
- MODEL: 153.875 linhas | 16 colunas

Toda a lógica documentada nas imagens pertence ao notebook de extração, nomeado como: 01_ingestion_peruvian_food_reviews.



3. Modelagem e Catálogo de Dados

A etapa de Modelagem e Catálogo de Dados tem como objetivo organizar os dados brutos extraídos em uma estrutura relacional otimizada para consultas analíticas. Abaixo, detalhamos a arquitetura relacional proposta, o catálogo de variáveis e a forma como isso é refletido no sistema de catálogo do Databricks.

a. Explicação da Modelagem de Dados

A modelagem do conjunto de dados "Peruvian Food Reviews" segue uma abordagem relacional (assemelhando-se a um *Star Schema* simplificado), onde as informações estão divididas entre tabelas de dimensão (entidades descritivas) e tabelas de fatos (eventos/transações).

Os relacionamentos definidos são:

- Tabela de Dimensão (Restaurants): Funciona como a entidade central de cadastro (mestre). A chave primária (PK) é a coluna Id.
- Tabela de Fatos (Reviews): Registra os eventos (avaliações feitas pelos usuários). Ela se conecta à tabela de restaurantes através da chave estrangeira (FK) Service,

que mapeia diretamente para a coluna Id da tabela de Restaurantes (Relacionamento 1 para N). A chave primária desta tabela é Id_review.

- Tabela de Extensão/Fatos (Models): Contém os metadados gerados por modelos de Machine Learning (análise de sentimentos e predição de estrelas). Ela possui um relacionamento de 1 para 1 com a tabela de Reviews, conectando-se através da chave Id_review.

b. Catálogo de Dados (Estrutura das Tabelas)

Abaixo está a transcrição do dicionário de dados organizado estruturalmente:

Tabela 1: Restaurants (Dimensão)

Esta tabela contém as informações descritivas, categóricas e geoespaciais dos estabelecimentos.

Variável	Descrição
Id	ID do restaurante (Chave Primária)
Name	Nome do restaurante
Tag	A categoria do restaurante
x, y	Informações geoespaciais e localização exata do restaurante
District	Distrito onde o restaurante está localizado
Direction	Endereço/distrito onde o restaurante está localizado
Stars	Média de estrelas do restaurante em todo o período
N_reviews	Número de avaliações do restaurante em todo o período
Min_Price	Preço mínimo no cardápio do restaurante
Max_Price	Preço máximo no cardápio do restaurante
Platform	Plataforma de onde as informações foram baixadas

Tabela 2: Reviews (Fato)

Esta tabela documenta a interação direta entre o cliente e o restaurante, incluindo o texto da avaliação e pontuações.

Variável	Descrição
Id_review	ID da avaliação (Chave Primária)
Id_nick	ID do usuário. Com isso é possível obter o link do perfil
Date	Data em que a avaliação foi escrita
Service	ID do restaurante. Conexão com a Tabela 1 (Chave Estrangeira)
Review	Conteúdo da avaliação. Descreve a satisfação do usuário
Title	Título da avaliação. Disponível apenas no Tripadvisor
Score	Pontuação na avaliação
Likes	Número de curtidas/votos na publicação
Platform	Plataforma de onde as informações foram baixadas

Tabela 3: Models (Extensão/Metadados)

Resultados de modelos de regressão e processamento de linguagem natural (NrcLexicon) aplicados ao conteúdo das avaliações.

Variável	Descrição
Id_review	ID da avaliação. Conexão com a Tabela 2 (Chave Estrangeira/Primária)
Positive	Probabilidade de a avaliação apresentar sentimento positivo
Negative	Probabilidade de a avaliação apresentar sentimento negativo
Anger	Probabilidade de a avaliação apresentar a emoção de raiva
Anticipation	Probabilidade de a avaliação apresentar a emoção de antecipação

Variável	Descrição
Disgust	Probabilidade de a avaliação apresentar a emoção de nojo/repulsa
Fear	Probabilidade de a avaliação apresentar a emoção de medo
Joy	Probabilidade de a avaliação apresentar a emoção de alegria
Sadness	Probabilidade de a avaliação apresentar a emoção de tristeza
Surprise	Probabilidade de a avaliação apresentar a emoção de surpresa
Stars_1	Probabilidade de a avaliação receber 1 estrela
Stars_2	Probabilidade de a avaliação receber 2 estrelas
Stars_3	Probabilidade de a avaliação receber 3 estrelas
Stars_4	Probabilidade de a avaliação receber 4 estrelas
Stars_5	Probabilidade de a avaliação receber 5 estrelas

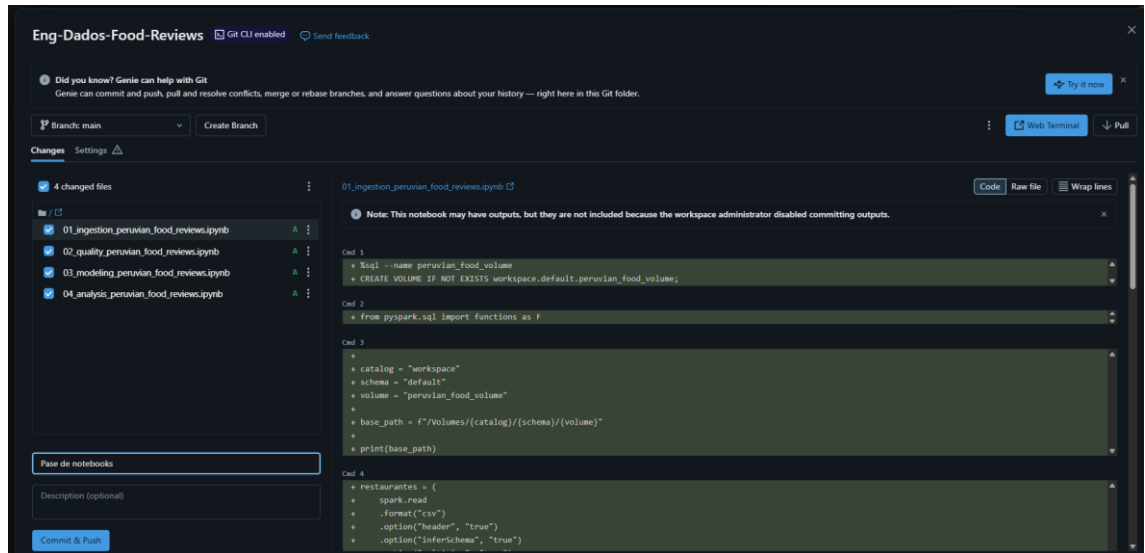
c. Implementação no Sistema de Catálogo (Referência às Screenshots)

Conforme evidenciado pelas capturas de tela do sistema a modelagem lógica descrita acima foi fisicamente implementada utilizando o Unity Catalog.

A arquitetura reflete uma estrutura de camadas (*Medallion Architecture*), o que pode ser confirmado pela lista de tabelas geradas no schema workspace.default:

- Camada Raw (Bruta): Tabelas `restaurantes_raw`, `reviews_raw`, `model_raw` (dados originais extraídos do Volume).
- Camada Clean (Prata/Transformada): Tabelas `restaurantes_clean`, `reviews_clean`, `model_clean` (dados tipificados, sem nulos ou inconsistências textuais).
- Camada Ouro (Modelada para Negócio): As tabelas finais materializadas seguindo exatamente a modelagem descrita acima, nomeadas no sistema de catálogo como `dim_restaurant` (Dimensão) e `fact_reviews` (Fatos).

4. Pipeline de Dados: Explique como organizou o processo de pipeline ETL, se tudo foi feito em um único notebook ou se ramificou e como ramificou. Adicione referência aos scripts disponibilizados no Github e screenshots que evidencie que essas tabelas foram salvas (persistidas) na plataforma de nuvem utilizada.



O processo de extração, transformação e carga (ETL) dos dados de avaliações de restaurantes peruanos foi organizado de forma modular na plataforma Databricks. Em vez de concentrar todo o processamento em um único notebook, o projeto foi dividido em quatro notebooks, cada um responsável por uma etapa específica do pipeline.

Essa organização permitiu separar as atividades de ingestão, tratamento da qualidade dos dados, modelagem e análise, facilitando a manutenção, validação e execução independente de cada etapa.

A estrutura geral do pipeline foi organizada da seguinte forma:

Dados de origem → Ingestão → Controle de qualidade → Modelagem → Análise

a) Notebook 01 – Ingestão dos dados

A primeira etapa foi implementada no notebook `01_ingestion_peruvian_food_reviews.ipynb`. Seu objetivo é realizar a ingestão dos datasets originais no ambiente Databricks.

Nesta etapa são carregados os dados referentes aos restaurantes e às avaliações, que posteriormente serão utilizados pelas etapas de limpeza, transformação e modelagem.

O código-fonte desta etapa está disponível no repositório público do projeto:

[01_ingestion_peruvian_food_reviews.ipynb – GitHub](#)

```

    Sep 18, 2026 (47s) 7
    restaurantes = spark.read.csv("/Volumes/workspace/default/peruvian_food_volume/restaurants.csv", header=True, inferSchema=True)
    print("RESTAURANTES")
    print("Linhas:", restaurantes.count())
    print("Colunas:", len(restaurantes.columns))
    print()

    reviews = spark.read.csv("/Volumes/workspace/default/peruvian_food_volume/reviews.csv", header=True, inferSchema=True)
    print("REVIEWS")
    print("Linhas:", reviews.count())
    print("Colunas:", len(reviews.columns))
    print()

    model = spark.read.csv("/Volumes/workspace/default/peruvian_food_volume/model.csv", header=True, inferSchema=True)
    print("MODEL")
    print("Linhas:", model.count())
    print("Colunas:", len(model.columns))

> See performance \(3\)

> 3 dataframes: model, restaurantes, reviews

RESTAURANTES
Linhas: 9871
Colunas: 13

REVIEWS
Linhas: 1345723
Colunas: 9

MODEL
Linhas: 153875
Colunas: 16

```

b) Notebook 02 – Qualidade e transformação dos dados

A segunda etapa foi implementada no notebook 02_quality_peruvian_food_reviews.ipynb.

Neste notebook são realizadas as operações de limpeza, validação e transformação dos dados. São aplicadas regras de qualidade para identificar registros inválidos, valores ausentes e informações que não atendem aos critérios definidos para o projeto.

Essa separação permite que os dados originais sejam mantidos independentes das transformações realizadas posteriormente.

Primeiro teste de qualidade: valores nulos

Agora vamos medir a completude, que é um dos critérios solicitados.

```

▶ ✓ Sep 12, 2026 (<1s) 6

def null_report(df, table_name):
    print(f"\n===== {table_name} =====")

    total = df.count()

    for column in df.columns:
        nulls = df.filter(
            F.col(column).isNull() | (F.trim(F.col(column).cast("string")) == "")
        ).count()

        percentage = (nulls / total) * 100

        print(f"{column}: {nulls} nulos/vacios ({percentage:.2f}%)")

▶ ✓ Sep 12, 2026 (26s) 7

null_report(restaurantes, "RESTAURANTES")
null_report(reviews, "REVIEWS")
null_report(model, "MODEL")

> See performance \(41\)
```

```
===== RESTAURANTES =====
id: 0 nulos/vacios (0.00%)
name: 0 nulos/vacios (0.00%)
tag: 251 nulos/vacios (2.54%)
x: 0 nulos/vacios (0.00%)
y: 0 nulos/vacios (0.00%)
district: 1082 nulos/vacios (10.96%)
IDDIST: 0 nulos/vacios (0.00%)
direction: 73 nulos/vacios (0.74%)
stars: 0 nulos/vacios (0.00%)
n_reviews: 0 nulos/vacios (0.00%)
min_price: 9335 nulos/vacios (94.57%)
max_price: 9349 nulos/vacios (94.71%)
platform: 20 nulos/vacios (0.20%)

===== REVIEWS =====
id_review: 23 nulos/vacios (0.00%)
review: 558874 nulos/vacios (41.53%)
title: 1256718 nulos/vacios (93.39%)
score: 108723 nulos/vacios (8.08%)
```

Segundo teste: duplicatas

Agora vamos verificar a unicidade.

▶ ✓ Sep 12, 2026 (1s)

9

```
print("==== DUPLICADOS RESTAURANTES ====")
```

```
    duplicados_restaurantes = (  
        restaurantes  
        .groupBy("id")  
        .count()  
        .filter(F.col("count") > 1)  
    )
```

```
    display(duplicados_restaurantes)
```

> [See performance \(1\)](#)

>  duplicados_restaurantes: pyspark.sql.connect.dataframe.DataFrame = [id: integer, count: long]

==== DUPLICADOS RESTAURANTES =====

Table  

1²₃ id

1²₃ count

No rows returned

```
print("===== DUPLICADOS REVIEWS =====")
```

```

duplicados_reviews = (
    reviews
    .groupBy("id_review")
    .count()
    .filter(F.col("count") > 1)
)

```

```
display(duplicados_reviews)
```

```
print("===== DUPLICADOS MODEL =====")
```

```

duplicados_model = (
    model
    .groupBy("id_review")
    .count()
    .filter(F.col("count") > 1)
)

```

```
display(duplicados_model)
```

> [See performance \(2\)](#)

> 2 dataframes: duplicados_reviews, duplicados_model

```
===== DUPLICADOS REVIEWS =====
```

Table  

	id_review	count
1	Buena atención.	27
2	Gracias"	62
3	La comida es rica	8
4	La carta es amplia	2
5	En si	3
6	El personal es muy amable	6
7	El menú es muy amplio	2
8	Local moderno	2
9	Con respecto a la comida	6
10	El personal es atento y amable.	3
11	Una ensalada	2
12	Respecto a la comida	12
13	Buena relación calidad precio."	2
14	Recomendado al 100%"	9
15	Los platos son abundantes	2

Terceiro teste: converter os campos numéricos

Este é um dos pontos importantes que descobrimos graças ao seu `printSchema()`.

▶ ✓ Sep 12, 2026 (<1s)

12

```
restaurantes_clean = (  
    restaurantes  
    .withColumn("id", F.col("id").cast("long"))  
    .withColumn("x", F.col("x").cast("double"))  
    .withColumn("y", F.col("y").cast("double"))  
    .withColumn("stars", F.col("stars").cast("double"))  
    .withColumn("n_reviews", F.col("n_reviews").cast("long"))  
    .withColumn("min_price", F.col("min_price").cast("double"))  
    .withColumn("max_price", F.col("max_price").cast("double"))  
)
```

> 📄 restaurantes_clean: pyspark.sql.connect.dataframe.DataFrame = [id: long, name: string ... 11 more fields]

▶ ✓ Sep 12, 2026 (<1s)

13

```
reviews_clean = (  
    reviews  
    .withColumn("score", F.col("score").cast("double"))  
    .withColumn("likes", F.col("likes").cast("long"))  
    .withColumn("service", F.col("service").cast("long"))  
)
```

> 📄 reviews_clean: pyspark.sql.connect.dataframe.DataFrame = [id_review: string, review: string ... 7 more fields]

▶ ✓ Sep 12, 2026 (<1s)

14

```
reviews_clean = (  
    reviews  
    .withColumn("score", F.col("score").cast("double"))  
    .withColumn("likes", F.col("likes").cast("long"))  
    .withColumn("service", F.col("service").cast("long"))  
)
```

> 📄 reviews_clean: pyspark.sql.connect.dataframe.DataFrame = [id_review: string, review: string ... 7 more fields]

Registros de modelo sem avaliação associada

▶ ✓ 2 minutes ago (3s)

```
total_model = model_clean.count()  
  
model_sin_review = (  
    model_clean  
    .join(  
        reviews_clean.select("id_review"),  
        on="id_review",  
        how="left"  
    )  
    .filter(reviews_clean.id_review.isNull())  
    .count()  
)  
  
print("Total registros model:", total_model)  
print("Registros de modelo sem avaliação associada:", model_sin_review)
```

> 📄 See performance (2)

Total registros model: 153875
Registros de modelo sem avaliação associada: 89554

Pontuações fora da faixa de 1 a 5

Just now (1s)30

```
print("==== Pontuações fora da faixa de 1 a 5 =====")

display(
  reviews_clean
    .filter(
      F.col("score_clean").isNotNull() &
      (
        (F.col("score_clean") < 1) |
        (F.col("score_clean") > 5)
      )
    )
    .select(
      "id_review",
      "score",
      "score_clean"
    )
    .limit(50)
)
```

> [See performance \(1\)](#)

==== Pontuações fora da faixa de 1 a 5 =====

Table +

	id_review	score	score_clean
1	Mejorar urgente el bar"	0	0
2	Lo recomiendo totalmente."	0	0
3	> Toda una experiencia diferente que hay que probar pero que una vez que lo has probado... ya está. No lo veo como un restaurante pa...	0	0
4	Gran experiencia!"	0	0
5	El cierre de un postre de chocolate con aceite de oliva sencillamente "maravilloso"!!!"	0	0
6	Tendré que regresar a probar los platos del menú."	0	0
7	> Los invito a que no se pierdan la oportunidad de visitar dicho restaurante cuanto antes. Les aseguro que no lo lamentaran. Por el cont...	0	0
8	> Al restaurante si me leen: Denle la oportunidad a los comensales de regular la cantidad de salsa en los platos. Y el salón podría estar ...	0	0
9	Felicitaciones!"	0	0
10	Ubicacion : Av Mariscal La Mar 463 Miraflores."	0	0
11	Bello y del!! Perfecta combinación."	0	0
12	Definitivamente regresaré para degustar estas exquisiteces."	0	0
13	Lo recomiendo a mill! El mejor restaurante al que he venido y la mejor experiencia que he tenido por lejos! Me encantó!"	0	0
14	De seguro vuelvo a pedir! mantengan siempre la buena calidad!"	0	0
15	La verdad es que no me impresionó."	0	0

50 rows

1.224s runtime

Preços negativos:

1 minute ago (1s)

32

```
print("==== PREÇOS NEGATIVOS =====")

display(
  restaurantes_clean
  .filter(
    (F.col("min_price_clean") < 0) |
    (F.col("max_price_clean") < 0)
  )
  .select(
    "id",
    "name",
    "min_price",
    "min_price_clean",
    "max_price",
    "max_price_clean"
  )
)
```

> [See performance \(1\)](#)

==== PREÇOS NEGATIVOS =====

Table +

	id	name	min_price	min_price_clean	max_price	max_price_clean
No rows returned						

0 rows

0.882s runtime

Min_price > Max_price

Just now (2s)

33

```
print("==== MIN_PRICE > MAX_PRICE =====")

display(
  restaurantes_clean
  .filter(
    F.col("min_price_clean").isNotNull() &
    F.col("max_price_clean").isNotNull() &
    (F.col("min_price_clean") > F.col("max_price_clean"))
  )
  .select(
    "id",
    "name",
    "min_price",
    "max_price",
    "min_price_clean",
    "max_price_clean"
  )
)
```

> [See performance \(1\)](#)

==== MIN_PRICE > MAX_PRICE =====

Table +

	id	name	min_price	max_price	min_price_clean	max_price_clean
1	1911	Mi Rico Chanco - D'MAC...	44	18	44	18
2	139979	Chifa Tumbalobos	36	26	36	26

2 rows

1.742s runtime

Avaliações com pontuação original não numérica

2 reviews ago (0)

34

Python

```
print("Total reviews:", reviews.count())

print(
    "reviews con score original no numérico:",
    reviews_clean
    .filter(
        F.expr("try_cast(trim(score) AS double)").isNull() &
        F.col("score").isNotNull()
    )
    .count()
)
```

See performance (2)

Checked for improvements

Total reviews: 1345723

Reviews con score original no numérico: 38941

Sep 12, 2024 (4)

35

Genie Code

```
display(
    reviews_clean
    .filter(
        F.expr("try_cast(trim(score) AS double)").isNull() &
        F.col("score").isNotNull()
    )
    .select(
        "id_review",
        "review",
        "title",
        "score",
        "likes",
        "service",
        "data",
        "platform"
    )
    .limit(10)
)
```

See performance (1)

Checked for improvements

Table

	id_review	review	title
1		Damos su bienvenida que envió a los restaurantes más recomendados del mundo	por el contrato
2		Algo que me gustó mucho fue que no te obligan a solo tener la opción del menú degustación	ellos disfrutaron de la carta y en del menú degustación con mandaje
3		No tienen sorpresas y de hecho si nos ofrecieron vinos. El menú de 7 platos tenía algunos logrados pero varios que realmente no. El f...	sería finto que haga clickeo a para suar. Pagamos 300 dólares lo cual es el trip
4		El local cuenta con techo alto y amplio	se podría decir minimalista
5		El servicio	es personalizado y muy amable. Personalmente
6		Fiel a mi estilo	me dejó guiar absolutamente. Inicieron con el pan de horta y aguije
7		La experiencia y en especial	hacen de esta visita
8		Cada miércoles pero variada y cada uno de los platos ofrece un espectáculo para los sentidos	oferta
9		R1289	la experiencia es increíble
10			

10 rows

0.770s runtime

Refreshed 2m ago

Nova extração com dados limpos



Just now (<1s)

37



```
reviews_clean = (  
  reviews  
  .withColumn(  
    "score_clean",  
    F.expr("try_cast(trim(score) AS double)")  
  )  
  .withColumn(  
    "likes_clean",  
    F.expr("try_cast(trim(likes) AS double)")  
  )  
  .withColumn(  
    "service_clean",  
    F.expr("try_cast(trim(service) AS double)")  
  )  
  .withColumn(  
    "restaurant_id",  
    F.expr("try_cast(service_clean AS long)")  
  )  
)
```

> reviews_clean: pyspark.sql.connect.dataframe.DataFrame = [id_review: string, review: string ... 11 more fields]



Sep 12, 2026 (2s)

38

```
reviews_invalidos = (  
  reviews_clean  
  .filter(  
    F.col("score_clean").isNull() |  
    ~F.col("score_clean").between(1, 5)  
  )  
)  
  
print("Registros excluidos por score:", reviews_invalidos.count())
```

> See performance (1)

Registros excluidos por score: 135791



▶ ✓ Sep 12, 2026 (<1s)

3

```
reviews_clean = (  
    reviews_clean  
    .filter(  
        F.col("score_clean").isNotNull() &  
        F.col("score_clean").between(1, 5)  
    )  
)
```

> reviews_clean: pyspark.sql.connect.dataframe.DataFrame = [id_review: string, review: string ... 11 more fields]



▶ ✓ Sep 12, 2026 (3s)

4

```
print("Reviews originales:", reviews.count())  
print("Reviews válidas:", reviews_clean.count())  
print("Reviews excluidas:", reviews.count() - reviews_clean.count())
```

> [See performance \(4\)](#)

Reviews originales: 1345723

Reviews válidas: 1209932

Reviews excluidas: 135791



▶ ✓ Sep 12, 2026 (2s)

4

```
display(  
    reviews_clean  
    .groupBy("score_clean")  
    .count()  
    .orderBy("score_clean")  
)
```

> [See performance \(1\)](#)

Table

	1.2 score_clean	1.2 count
1	1	58830
2	2	44277

```
▶ ✓ Sep 12, 2026 (6s)

reviews_clean.write \
  .mode("overwrite") \
  .format("delta") \
  .saveAsTable("workspace.default.reviews_clean")

> See performance \(1\)
```

```
▶ ✓ Sep 12, 2026 (3s)

restaurantes_clean.write \
  .mode("overwrite") \
  .format("delta") \
  .saveAsTable("workspace.default.restaurantes_clean")

> See performance \(1\)
```

```
< >

▶ ✓ Sep 12, 2026 (5s)

model_clean.write \
  .mode("overwrite") \
  .format("delta") \
  .saveAsTable("workspace.default.model_clean")

> See performance \(1\)
```

```
▶ ✓ Sep 12, 2026 (3s)

print("Reviews originaes:", reviews.count())
print("Reviews válidas:", reviews_clean.count())
print("Reviews excluídas:", reviews.count() - reviews_clean.count())

> See performance \(4\)
```

O código desta etapa está disponível em:

[02_quality_peruvian_food_reviews.ipynb – GitHub](#)

c) Notebook 03 – Modelagem dos dados

A terceira etapa corresponde ao notebook 03_modeling_peruvian_food_reviews.ipynb.

Após o processo de limpeza e transformação, os dados são utilizados para construir o modelo de dados do projeto. Nesta etapa são organizadas as estruturas necessárias para realizar as consultas e análises posteriores.

Criar dim_restaurant

Nossa dimensão de restaurantes conterá informações descritivas do estabelecimento.

Primeiro, criaremos uma versão com as colunas limpas.

```

▶ Sep 14, 2026 (1s) 4

dim_restaurant = (
    restaurantes_clean
    .select(
        F.col("id_clean").alias("restaurant_id"),
        F.trim(F.col("name")).alias("restaurant_name"),
        F.trim(F.col("tag")).alias("tags"),
        F.col("x_clean").alias("latitude"),
        F.col("y_clean").alias("longitude"),
        F.upper(F.trim(F.col("district"))).alias("district"),
        F.trim(F.col("IDDIST")).alias("district_id"),
        F.trim(F.col("direction")).alias("address"),
        F.col("stars_clean").alias("restaurant_stars"),
        F.col("n_reviews_clean").alias("restaurant_review_count"),
        F.col("min_price_clean").alias("min_price"),
        F.col("max_price_clean").alias("max_price"),
        F.trim(F.col("platform")).alias("restaurant_platform")
    )
    .filter(F.col("restaurant_id").isNotNull())
    .dropDuplicates(["restaurant_id"])
)

> dim_restaurant: pyspark.sql.connect.dataframe.DataFrame = [restaurant_id: long, restaurant_name: string ... 11 more fields]
```

Agora, adicionamos duas variáveis úteis para a análise:

```

▶ Sep 14, 2026 (1s) 6

dim_restaurant = (
    dim_restaurant
    .withColumn(
        "average_price",
        F.when(
            F.col("min_price").isNotNull() &
            F.col("max_price").isNotNull(),
            (F.col("min_price") + F.col("max_price")) / 2
        )
    )
    .withColumn(
        "price_range",
        F.when(F.col("average_price").isNull(), "Unknown")
        .when(F.col("average_price") < 50, "Low")
        .when(F.col("average_price") < 100, "Medium")
        .when(F.col("average_price") < 200, "High")
        .otherwise("Premium")
    )
)

> dim_restaurant: pyspark.sql.connect.dataframe.DataFrame = [restaurant_id: long, restaurant_name: string ... 13 more fields]
```

Criar fact_reviews

Esta será a nossa tabela principal.

```
fact_reviews = (
    reviews_clean
    .select(
        F.trim(F.col("id_review")).alias("review_id"),
        F.col("restaurant_id").alias("restaurant_id"),
        F.trim(F.col("review")).alias("review_text"),
        F.trim(F.col("title")).alias("review_title"),
        F.col("score_clean").alias("score"),
        F.col("likes_clean").alias("likes"),
        F.trim(F.col("id_nick")).alias("user_nickname"),
        F.trim(F.col("date")).alias("review_date_text"),
        F.trim(F.col("platform")).alias("review_platform")
    )
    .filter(
        F.col("review_id").isNotNull() &
        F.col("restaurant_id").isNotNull()
    )
)
```

> fact_reviews: pyspark.sql.connect.dataframe.DataFrame = [review_id: string, restaurant_id: long ... 7 more fields]

Agora, adicionaremos categorias de satisfação:

```
fact_reviews = (
    fact_reviews
    .withColumn(
        "satisfaction_level",
        F.when(F.col("score") <= 2, "Low")
        .when(F.col("score") == 3, "Neutral")
        .otherwise("High")
    )
)
```

> fact_reviews: pyspark.sql.connect.dataframe.DataFrame = [review_id: string, restaurant_id: long ... 8 more fields]

```
display(
    fact_reviews
    .groupBy("satisfaction_level")
    .count()
)
```

> [See performance \(1\)](#)

	^A _C satisfaction_level	¹ ₃ count
1	High	949940
2	Neutral	156729
3	Low	101080

Integrar as emoções do modelo

Agora vamos unir:

fact_reviews + model_clean

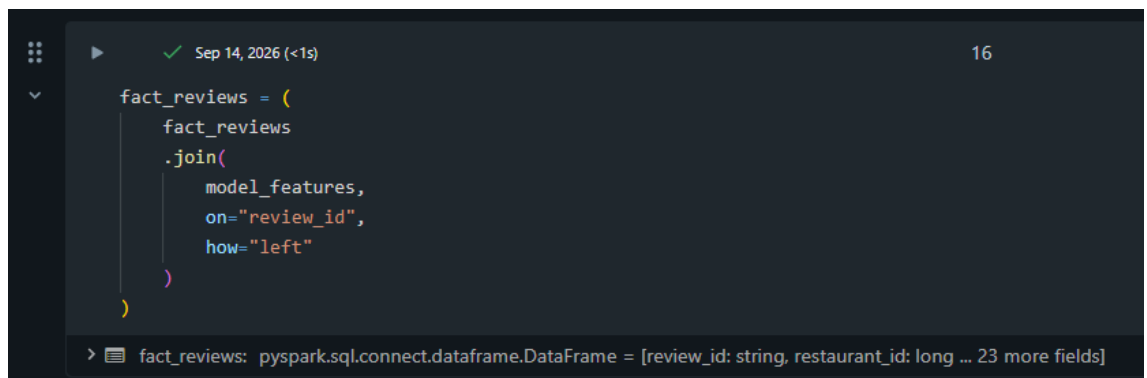


```
model_features = (
    model_clean
    .select(
        F.trim(F.col("id_review")).alias("review_id"),
        "anger",
        "anticipation",
        "disgust",
        "fear",
        "joy",
        "sadness",
        "surprise",
        "trust",
        "negative",
        "positive",
        "stars_1",
        "stars_2",
        "stars_3",
        "stars_4",
        "stars_5"
    )
)
```

> model_features: pyspark.sql.connect.dataframe.DataFrame = [review_id: string, anger: double ... 14 more fields]

Por que usamos left?

Porque não queremos perder as avaliações válidas que não aparecem em model.csv.



```
fact_reviews = (
    fact_reviews
    .join(
        model_features,
        on="review_id",
        how="left"
    )
)
```

> fact_reviews: pyspark.sql.connect.dataframe.DataFrame = [review_id: string, restaurant_id: long ... 23 more fields]

Criar uma variável de sentimento

Vamos aproveitar positive e negative.

Sep 14, 2026 (<1s)

18

```
fact_reviews = (  
  fact_reviews  
  .withColumn(  
    "sentiment",  
    F.when(  
      F.col("positive").isNull() |  
      F.col("negative").isNull(),  
      "Not available"  
    )  
    .when(F.col("positive") > F.col("negative"), "Positive")  
    .when(F.col("negative") > F.col("positive"), "Negative")  
    .otherwise("Neutral")  
  )  
)
```

>  fact_reviews: pyspark.sql.connect.dataframe.DataFrame = [review_id: string, restaurant_id: long ... 24 more fields]

Sep 14, 2026 (2s)

19

```
display(  
  fact_reviews  
  .groupBy("sentiment")  
  .count()  
)
```

> [See performance \(1\)](#)

Table  

	sentiment	count
1	Not available	1156338
2	Neutral	5162
3	Negative	4404
4	Positive	41845

Identificar a emoção predominante

Isso tornará nosso projeto mais interessante.

```

emotion_columns = [
    "anger",
    "anticipation",
    "disgust",
    "fear",
    "joy",
    "sadness",
    "surprise",
    "trust"
]

fact_reviews = (
    fact_reviews
    .withColumn(
        "max_emotion_score",
        F.greatest(
            F.col("anger"),
            F.col("anticipation"),
            F.col("disgust"),
            F.col("fear"),
            F.col("joy"),
            F.col("sadness"),
            F.col("surprise"),
            F.col("trust")
        )
    )
    .withColumn(
        "dominant_emotion",
        F.when(F.col("max_emotion_score").isNull(), "Not available")
        .when(F.col("anger") == F.col("max_emotion_score"), "anger")
        .when(F.col("anticipation") == F.col("max_emotion_score"), "anticipation")
        .when(F.col("disgust") == F.col("max_emotion_score"), "disgust")
        .when(F.col("fear") == F.col("max_emotion_score"), "fear")
        .when(F.col("joy") == F.col("max_emotion_score"), "joy")
        .when(F.col("sadness") == F.col("max_emotion_score"), "sadness")
        .when(F.col("surprise") == F.col("max_emotion_score"), "surprise")
        .when(F.col("trust") == F.col("max_emotion_score"), "trust")
        .otherwise("Not available")
    )
    .drop("max_emotion_score")
)

```



```
▶ Sep 14, 2026 (2s)

reviews_without_restaurant = (
    fact_reviews
    .join(
        dim_restaurant.select("restaurant_id"),
        on="restaurant_id",
        how="left_anti"
    )
)

print(
    "Reviews válidas sin restaurante:",
    reviews_without_restaurant.count()
)

> See performance \(1\)

> reviews_without_restaurant: pyspark.sql.connect.dataframe.DataFrame = [restaurant_id: long, review_id: string ... 25 more fields]
Reviews válidas sin restaurante: 0

▶ Sep 14, 2026 (4s)

print("Total fact reviews:", fact_reviews.count())

print(
    "Reviews con información de sentimiento:",
    fact_reviews.filter(
        F.col("positive").isNotNull()
    ).count()
)

print(
    "Reviews sin información de sentimiento:",
    fact_reviews.filter(
        F.col("positive").isNull()
    ).count()
)

> See performance \(3\)

Total fact reviews: 1207749
Reviews con información de sentimiento: 51411
Reviews sin información de sentimiento: 1156338
```

Salvar as tabelas finais

Primeiro, a dimensão:

```
▶ ✓ Sep 14, 2026 (8s)
dim_restaurant.write \
  .format("delta") \
  .mode("overwrite") \
  .saveAsTable("workspace.default.dim_restaurant")
> See performance \(1\)
```

```
▶ ✓ Sep 14, 2026 (7s)
fact_reviews.write \
  .format("delta") \
  .mode("overwrite") \
  .saveAsTable("workspace.default.fact_reviews")
> See performance \(1\)
```

```
▶ ✓ Sep 14, 2026 (1s)
print("DIM_RESTAURANT:", spark.table(
  "workspace.default.dim_restaurant"
).count())

print("FACT_REVIEWS:", spark.table(
  "workspace.default.fact_reviews"
).count())
> See performance \(2\)
```

DIM_RESTAURANT: 9871
FACT_REVIEWS: 1207749

```

    reviews_without_restaurant = (
        fact_reviews
        .join(
            dim_restaurant.select("restaurant_id"),
            on="restaurant_id",
            how="left_anti"
        )
    )

    print(
        "Reviews válidas sin restaurante:",
        reviews_without_restaurant.count()
    )

> See performance \(1\)

> reviews_without_restaurant: pyspark.sql.connect.dataframe.DataFrame = [restaurant_id: long, review_id: string ... 25 more fields]
Reviews válidas sin restaurante: 0

    print("Total fact reviews:", fact_reviews.count())

    print(
        "Reviews con información de sentimiento:",
        fact_reviews.filter(
            F.col("positive").isNotNull()
        ).count()
    )

    print(
        "Reviews sin información de sentimiento:",
        fact_reviews.filter(
            F.col("positive").isNull()
        ).count()
    )

> See performance \(3\)

Total fact reviews: 1207749
Reviews con información de sentimiento: 51411
Reviews sin información de sentimiento: 1156338

```

O código-fonte está disponível em:

[03_modeling_peruvian_food_reviews.ipynb – GitHub](#)

d) Notebook 04 – Análise dos dados

A última etapa foi implementada no notebook `04_analysis_peruvian_food_reviews.ipynb`.

Este notebook utiliza as estruturas produzidas nas etapas anteriores para realizar as consultas analíticas, agregações e geração dos resultados utilizados no projeto.

O código está disponível em:

[04_analysis_peruvian_food_reviews.ipynb – GitHub](#)

Organização do pipeline

Dessa forma, o pipeline foi dividido em quatro notebooks independentes, seguindo uma sequência lógica:

Ordem	Notebook	Principal finalidade
1	01_ingestion_peruvian_food_reviews.ipynb	Ingestão dos dados originais
2	02_quality_peruvian_food_reviews.ipynb	Limpeza, validação e transformação
3	03_modeling_peruvian_food_reviews.ipynb	Modelagem e estruturação dos dados
4	04_analysis_peruvian_food_reviews.ipynb	Consultas e análise dos dados

Portanto, o projeto não foi desenvolvido em um único notebook. O processo foi ramificado em quatro notebooks especializados, nos quais cada etapa utiliza os dados produzidos pela etapa anterior.

Essa organização segue o princípio de separação de responsabilidades, permitindo que cada notebook tenha uma finalidade específica e facilitando a manutenção e a reprodução do pipeline.

5. Qualidade de Dados: Quais problemas foram detectados e como resolveu cada um deles, que transformações foram feitas.

A etapa de qualidade dos dados foi realizada no notebook 02_quality_peruvian_food_reviews.ipynb, dedicado à identificação, tratamento e transformação dos problemas encontrados nos dados originais. O notebook possui 1.408 linhas de código e concentra as principais regras de limpeza e validação aplicadas ao conjunto de dados.

Durante a análise dos dados foram identificados problemas relacionados principalmente a registros inválidos, valores ausentes, campos textuais sem informação útil e inconsistências nos dados de avaliações. Para cada problema foi definida uma regra específica de tratamento, buscando preservar somente os registros adequados para as etapas posteriores do pipeline.

Problemas identificados e soluções aplicadas:

a) Registros inválidos nas avaliações

Um dos principais problemas encontrados ocorreu no dataset de avaliações (REVIEWS). Alguns registros apresentavam informações inconsistentes no campo relacionado à pontuação da avaliação.

Inicialmente foram identificados 1.345.723 registros no conjunto original. Após a aplicação das regras de validação, foram considerados 1.209.932 registros válidos, enquanto 135.791 registros foram excluídos por não atenderem aos critérios definidos para a pontuação.

A solução adotada foi realizar uma validação do campo de pontuação e excluir os registros que apresentavam valores incompatíveis com o domínio esperado.

Dessa forma:

REVIEWS	original:	1.345.723	registros
REVIEWS	válidos:	1.209.932	registros
Registros excluídos: 135.791 registros			

Essa transformação evita que avaliações com pontuações inválidas influenciem os indicadores e análises realizadas posteriormente.

b) Valores nulos ou vazios no campo tag

Também foram encontrados restaurantes cujo campo tag estava vazio ou apresentava valor nulo.

Como o campo tag é utilizado para caracterizar ou classificar os restaurantes, registros sem essa informação não apresentavam utilidade para determinadas análises realizadas no projeto.

A solução foi aplicar filtros para identificar os registros com tag nulo ou vazio e considerar somente os restaurantes que possuíam uma informação válida nesse campo nas análises que dependiam dessa característica.

Essa transformação permitiu evitar que restaurantes sem classificação fossem utilizados indevidamente nos agrupamentos e indicadores.

c) Valores não disponíveis

Outro problema identificado foi a existência de valores textuais representando ausência de informação, como "not available".

Esses valores não representam uma categoria real, mas sim a ausência de informação. Por esse motivo, foram tratados como valores sem informação para as análises que utilizavam esses atributos.

A transformação consistiu em excluir ou desconsiderar esses valores durante os filtros e agregações, evitando que "not available" fosse interpretado como uma categoria válida.

d) Informações ausentes relacionadas à localização

Foram identificados também registros sem uma informação válida de distrito ou localização.

Como o distrito é utilizado posteriormente para análises geográficas e comparações de desempenho dos restaurantes, registros sem uma identificação válida não poderiam ser corretamente associados a um distrito.

Por isso, foram considerados apenas os registros que possuíam valores válidos para o distrito nas análises correspondentes.

Essa transformação permitiu construir posteriormente indicadores como quantidade de restaurantes, quantidade de avaliações e desempenho por distrito sem considerar categorias vazias ou inválidas.

e) Transformações realizadas

Além da eliminação dos registros inválidos, foram realizadas transformações para preparar os dados para as etapas de modelagem e análise.

As principais transformações foram:

Problema identificado	Tratamento realizado	Objetivo
Pontuações inválidas nas avaliações	Validação e exclusão dos registros inválidos	Garantir consistência das avaliações
Registros com tag nulo ou vazio	Filtragem dos registros	Evitar restaurantes sem classificação nas análises correspondentes
Valores "not available"	Exclusão/desconsideração nas análises	Evitar categorias artificiais
Distrito nulo ou vazio	Filtragem	Garantir análises geográficas consistentes
Dados inconsistentes	Aplicação de regras de validação	Melhorar a confiabilidade dos dados
Registros não adequados para análise	Remoção após validação	Evitar que dados inválidos afetem os resultados

Resultado do processo de qualidade:

Após a aplicação das regras de qualidade, foi obtido um conjunto de dados mais consistente para ser utilizado nas etapas seguintes do pipeline.

O principal resultado quantitativo ocorreu no dataset de avaliações: dos 1.345.723 registros originais, 1.209.932 foram considerados válidos, enquanto 135.791 foram excluídos durante o processo de qualidade.

Isso significa que aproximadamente 89,91% dos registros originais foram preservados para utilização posterior, enquanto aproximadamente 10,09% foram removidos por apresentarem problemas relacionados à qualidade dos dados.

A etapa de qualidade, portanto, não consistiu apenas na remoção de registros. Foram aplicadas regras de validação, filtragem e preparação dos atributos, garantindo que os dados utilizados na etapa de modelagem apresentassem maior consistência e fossem adequados aos objetivos analíticos do projeto.

6. Análise de Dados:

A etapa de análise de dados foi realizada no notebook 04_analysis_peruvian_food_reviews.ipynb, desenvolvido em Python utilizando PySpark, Pandas e Matplotlib. O notebook utiliza como fonte principal as tabelas dim_restaurant e fact_reviews, que foram geradas nas etapas anteriores do pipeline.

O objetivo desta etapa foi transformar os dados processados em informações úteis para responder às perguntas de negócio definidas para o projeto. Para isso, foram utilizadas operações de JOIN, GROUP BY, filtros, agregações, médias, contagens, cálculos percentuais e comparações, além da geração de gráficos para facilitar a interpretação dos resultados.

O notebook foi estruturado em cinco perguntas principais.

Pergunta 1 – Quais são os restaurantes mais populares e mais bem avaliados?

Para responder a essa pergunta, foi realizada uma combinação entre as tabelas fact_reviews e dim_restaurant utilizando o identificador do restaurante.

Foram calculadas duas métricas principais:

- Número de avaliações: utilizado como indicador de popularidade;
- Pontuação média: utilizada como indicador da percepção de qualidade.

A consulta agrupa os dados por restaurante e calcula a quantidade de avaliações e a média das pontuações. Em seguida, os restaurantes são ordenados pela quantidade de avaliações para identificar aqueles com maior volume de opiniões.

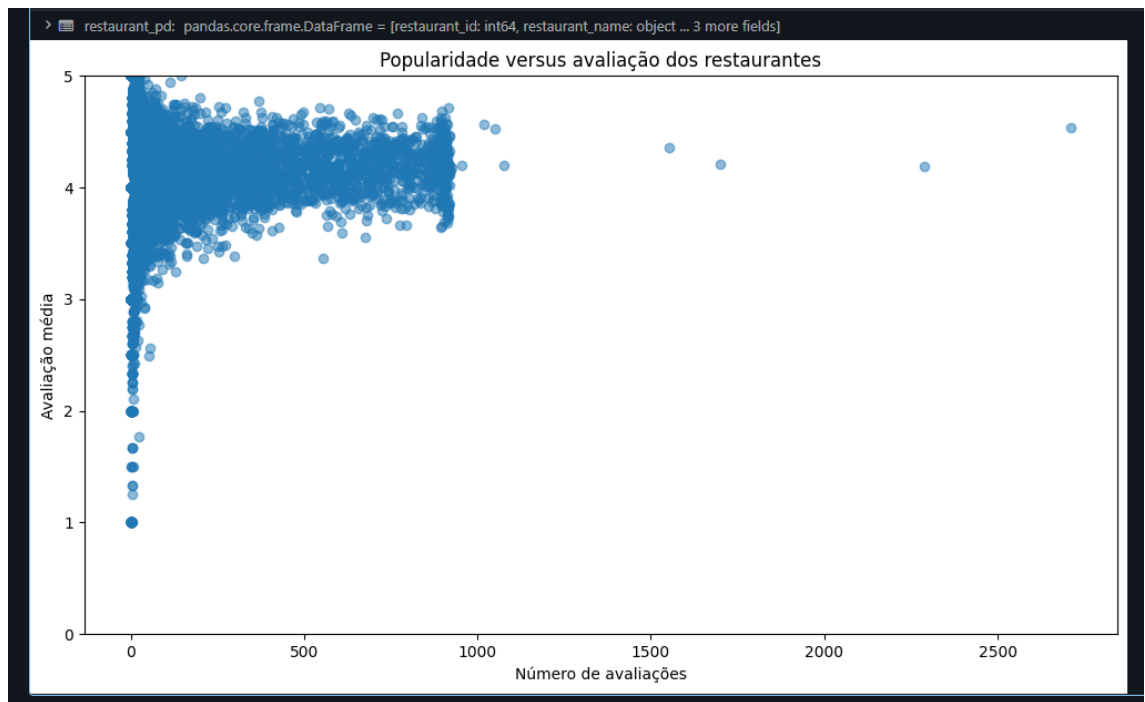
Também foi criado um gráfico de dispersão relacionando:

Número de avaliações × Pontuação média

Essa visualização permite identificar diferentes perfis de restaurantes, por exemplo, restaurantes com muitas avaliações e alta pontuação, restaurantes populares porém com menor pontuação e restaurantes com poucas avaliações mas pontuação elevada.

Table +					
	1.2 restaurant_id	1.1 restaurant_name	1.0 district	1.3 num_reviews	1.2 average_score
1	124291	Cebicheria La Mar	MIRAFLORES	2710	4.54
2	242558	Mangos	MIRAFLORES	2289	4.19
3	126618	Tanta	MIRAFLORES	1699	4.21
4	221166	Cala Restaurante	BARRANCO	1553	4.36
5	147048	Parque Próceres de la Independencia	JESUS MARIA	1077	4.2
6	124060	Cala	BARRANCO	1049	4.53
7	246459	Panchita - Miraflores	MIRAFLORES	1019	4.57
8	147308	El Tarwi - La Victoria	LA VICTORIA	953	4.2
9	143149	Mesa de Piedra - Restaurante Cultural	CIENEGUILLA	926	4.18
10	174285	Papa John's	LOS OLIVOS	922	4.15
11	8538	BIMBO		920	4.11
12	87158	Caldo de Gallina Aramburu	SURQUILLO	920	4.02
13	162692	Haití Café - Bar - Restaurant	MIRAFLORES	919	4.46
14	89263	La Hacienda Monterrico Grande	LA MOLINA	918	4.33
15	195070	San Antonio - Chacarilla	SAN BORJA	918	4.47

20 rows 12.750s runtime



Pergunta 2 – Quais distritos concentram a maior popularidade e qualidade?

A segunda análise foi realizada no nível dos distritos.

Para cada distrito foram calculados:

- quantidade de restaurantes;
- quantidade de avaliações;
- pontuação média.

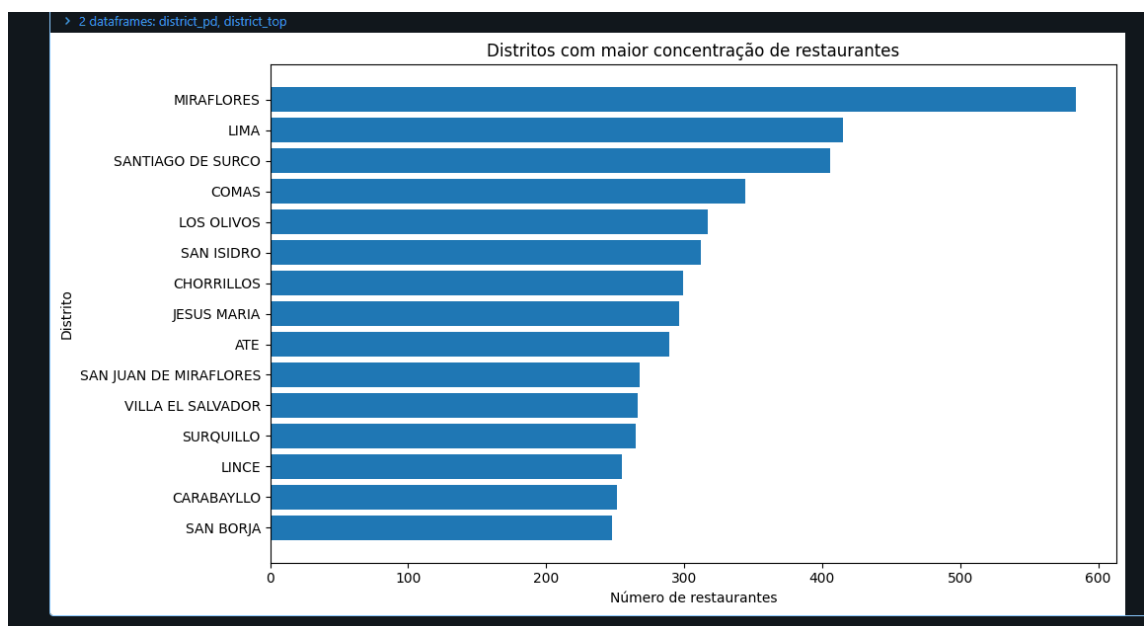
Antes da agregação, foram removidos registros cujo distrito fosse nulo ou vazio. Além disso, foi estabelecido um mínimo de 100 avaliações por distrito, evitando que localidades com quantidade muito pequena de avaliações distorcessem a análise.

Posteriormente, os distritos foram ordenados pelo número de avaliações e foi produzido um gráfico com os distritos que apresentam maior concentração de restaurantes.

Essa análise permite compreender a distribuição geográfica da atividade dos restaurantes e identificar os distritos que concentram maior volume de avaliações.

	^{1.0} district	^{1.2} num_restaurants	^{1.2} num_reviews	^{1.2} average_score
1	MIRAFLORES	584	98103	4.27
2	SANTIAGO DE SURCO	406	70465	4.18
3	JESUS MARIA	296	59196	4.18
4	SAN ISIDRO	312	55154	4.27
5	LOS OLIVOS	317	52820	4.16
6	SAN MIGUEL	244	52586	4.19
7	SAN BORJA	248	52034	4.22
8	LIMA	415	50131	4.15
9	LA MOLINA	241	49182	4.16
10	BARRANCO	238	39623	4.32
11	LINCE	255	37917	4.15
12	PUEBLO LIBRE	219	37829	4.17
13	CHORRILLOS	299	36536	4.19
14	COMAS	344	32110	4.1
15	LA VICTORIA	235	31449	4.12

43 rows 2.705s runtime



Pergunta 3 – Quais restaurantes se destacam em relação à média do seu distrito?

Nesta análise foi realizada uma comparação entre o desempenho individual de cada restaurante e a média de pontuação do respectivo distrito.

Primeiramente, foi calculada a pontuação média de cada distrito. Em seguida, essa informação foi associada aos restaurantes pertencentes a cada distrito.

Foi criada a métrica:

Diferença em relação ao distrito = Pontuação média do restaurante – Pontuação média do distrito

Também foi aplicado um filtro mínimo de 100 avaliações por restaurante, buscando evitar conclusões baseadas em um número muito pequeno de opiniões.

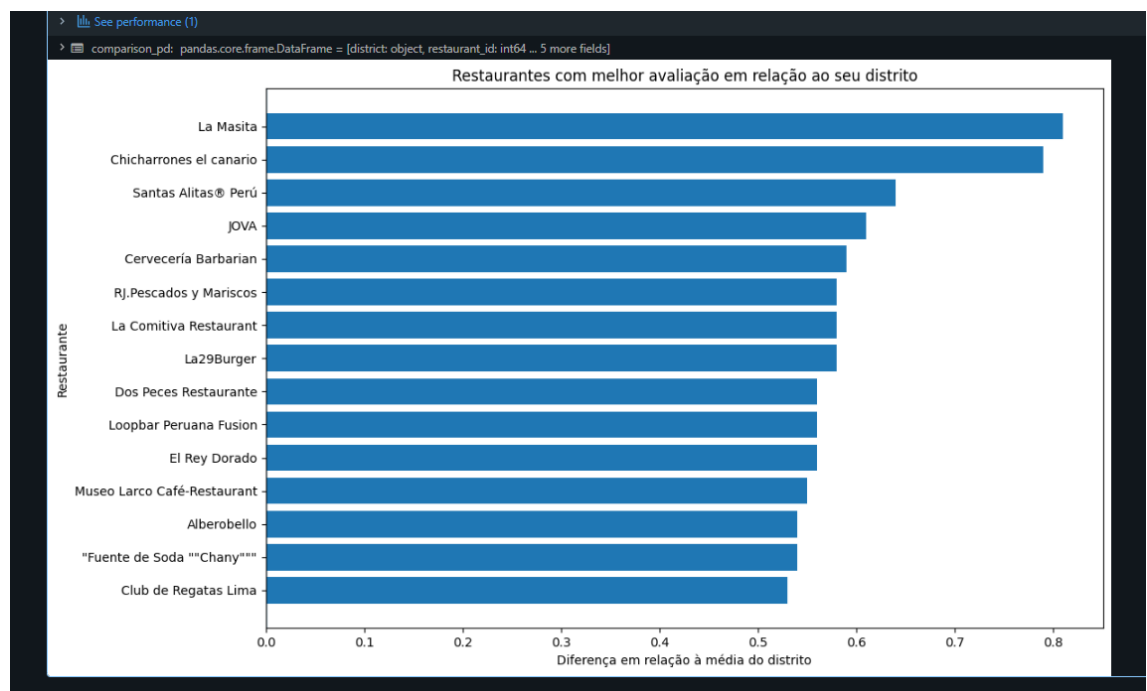
Assim, um valor positivo indica que o restaurante possui uma pontuação superior à média do seu distrito. Por exemplo, uma diferença de +0,50 significa que a pontuação média do restaurante está 0,50 ponto acima da média do distrito.

Foi também produzido um gráfico de barras para visualizar os restaurantes que apresentam maior diferença positiva em relação à média de sua região.

> 2 dataframes: district_avg, restaurant_vs_district

	district	restaurant_id	restaurant_name	num_reviews	average_score	district_average_score	difference_from_district
1	SAN MIGUEL	155753	La Masita	145	5	4.185125318525843	
2	VILLA EL SALVADOR	182147	Chicharrones el canario	114	4.94	4.14814631979069	
3	SAN JUAN DE MIRAFLORES	15170	Santas Alitas® Perú	118	4.7	4.0646525679758305	
4	PUEBLO LIBRE	45748	JOVA	369	4.78	4.173147585185968	
5	LURIGANCHO	180616	Cervecería Barbarian	122	4.64	4.04656583263831	
6	SANTIAGO DE SURCO	81745	La Comitiva Restaurant	154	4.76	4.175973887745689	
7	COMAS	100921	La29Burger	165	4.68	4.103145437558393	
8	LA VICTORIA	192769	RJ.Pescados y Mariscos	146	4.7	4.123692327260009	
9	SAN MARTIN DE PORRES	44591	El Rey Dorado	251	4.61	4.05338078291815	
10	CHORRILLOS	15469	Dos Peces Restaurante	132	4.75	4.191892927523538	
11	CARABAYLLO	92157	Loopbar Peruana Fusion	112	4.66	4.09563360227812	
12	PUEBLO LIBRE	279219	Museo Larco Café-Restaurant	544	4.72	4.173147585185968	
13	LA VICTORIA	48815	"Fuente de Soda ""Chany""	184	4.66	4.123692327260009	
14	SAN MIGUEL	32444	Alberobello	154	4.73	4.185125318525843	

20 rows 3.832s runtime Refreshed Last wk.



Um valor de:

+0,50

significa que o restaurante está 0,50 pontos acima da média do seu distrito.

Isso permite encontrar restaurantes que realmente se destacam em relação aos seus concorrentes locais.

Pergunta 4 – Quais emoções caracterizam as opiniões positivas e negativas?

A quarta análise utiliza as informações de sentimento e emoção predominante presentes na tabela fact_reviews.

Foram considerados somente registros que possuem valores válidos para:

- sentiment;
- dominant_emotion.

Os valores nulos e aqueles classificados como "Not available" foram desconsiderados na análise.

Posteriormente, os registros foram agrupados por:

Sentimento × Emoção predominante

e foi calculada a quantidade de avaliações pertencente a cada combinação.

Com esses resultados foi construído um gráfico que permite comparar a distribuição das emoções de acordo com o sentimento identificado nas avaliações.

Essa análise permite compreender não apenas se as avaliações são positivas ou negativas, mas também quais emoções predominam dentro de cada classificação de sentimento.

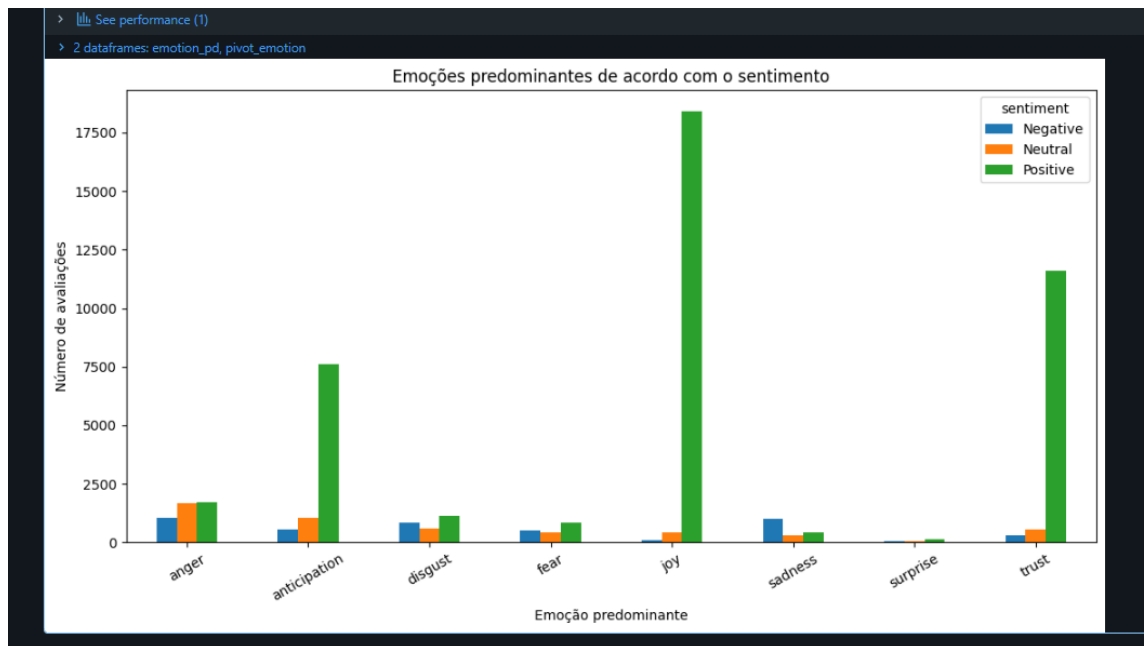
> [See performance \(1\)](#)

> `emotion_sentiment: pyspark.sql.connect.dataframe.DataFrame = [sentiment: string, dominant_emotion: string ... 1 more field]`

Table +

	sentiment	dominant_emotion	num_reviews
1	Negative	anger	1038
2	Negative	sadness	1029
3	Negative	disgust	839
4	Negative	anticipation	561
5	Negative	fear	495
6	Negative	trust	297
7	Negative	joy	105
8	Negative	surprise	40
9	Neutral	anger	1691
10	Neutral	anticipation	1040
11	Neutral	disgust	616
12	Neutral	trust	573
13	Neutral	joy	448
14	Neutral	fear	428
15	Neutral	sadness	311

24 rows | 1.704s runtime



Pergunta 5 – Quais restaurantes combinam alta popularidade com alta satisfação?

A quinta pergunta foi definida como a análise de destaque do projeto.

Para cada restaurante foram calculadas três métricas:

- número total de avaliações;
- pontuação média;
- percentual de avaliações positivas.

O percentual de avaliações positivas foi calculado pela seguinte fórmula:

Percentual de avaliações positivas = $(\text{Avaliações positivas} / \text{Total de avaliações}) \times 100$

Para aumentar a confiabilidade da comparação, foram considerados restaurantes com pelo menos 100 avaliações. Depois, essas métricas foram associadas ao nome e ao distrito do restaurante.

Foi construído um gráfico de dispersão relacionando:

Número de avaliações $\times$ Percentual de avaliações positivas

A interpretação do gráfico permite identificar a região superior direita, que representa restaurantes que apresentam simultaneamente:

Alta popularidade + Alta satisfação

Essa visualização fornece uma forma objetiva de identificar restaurantes que possuem grande volume de avaliações e, ao mesmo tempo, elevada proporção de avaliações positivas.

See performance (1)

restaurant_satisfaction: pyspark.sql.connect.dataframe.DataFrame = [restaurant_id: long, num_reviews: long ... 5 more fields]

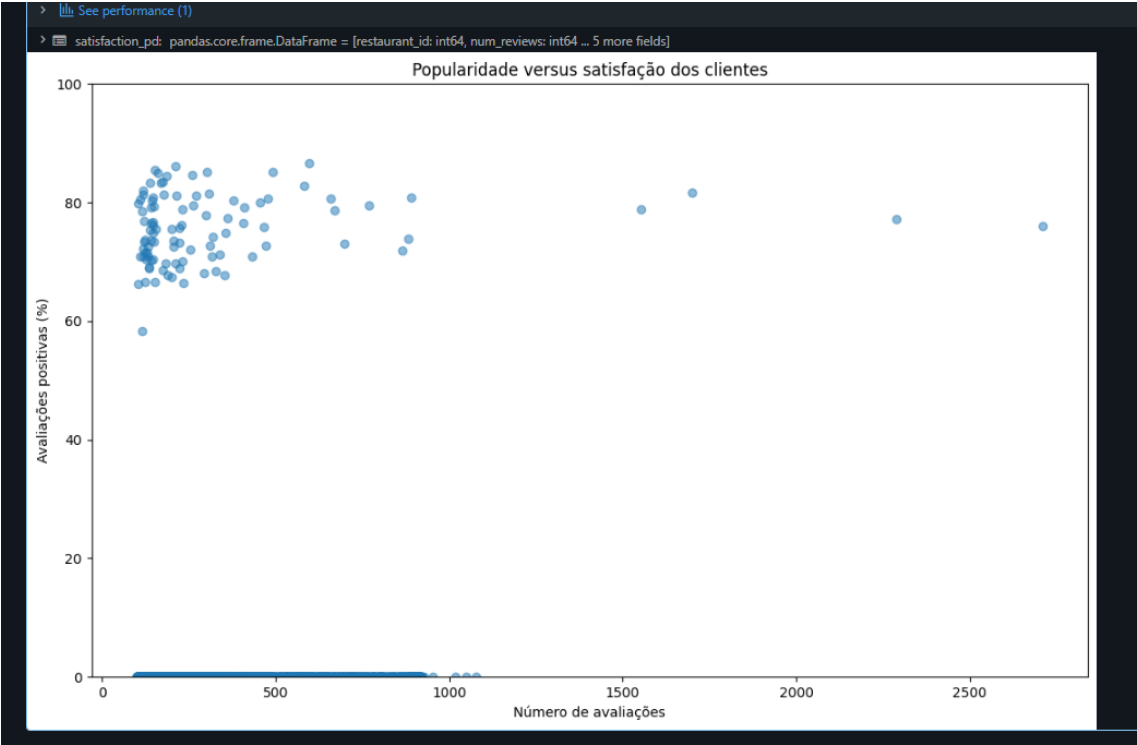
Table

	restaurant_id	num_reviews	average_score	positive_reviews	positive_percentage	restaurant_name	district
1	67121	596	4.42	516	86.58	Puquina Qocha	LIMA
2	216023	210	4.28	181	86.19	Restaurante Las Tejas	MIRAFLORES
3	243588	152	4.05	130	85.53	Restaurante árabe tierra santa	MIRAFLORES
4	72471	491	4.66	418	85.13	Maras Restaurante	SAN ISIDRO
5	171382	302	4.05	257	85.1	Cafe de la Paz	MIRAFLORES
6	181709	160	4.59	136	85	Tzuru Restaurante Nikkei	SAN ISIDRO
7	144186	260	4.68	220	84.62	Aji 555 Real Thai Cuisine	SAN BORJA
8	29667	186	4.72	157	84.41	Ventarrón Restaurante Chiclayano & Barra	BARRANCO
9	294925	175	4.14	146	83.43	Donatello	MIRAFLORES
10	138180	138	4.64	115	83.33	Chabuca Restaurant	SAN ISIDRO
11	210525	168	4.46	140	83.33	Delphos Bistro	SAN ISIDRO
12	54216	581	4.59	481	82.79	Cosme	SAN ISIDRO
13	76722	117	4.47	96	82.05	Restaurante Arlotia	BARRANCO
14	126618	1699	4.21	1389	81.75	Tanta	MIRAFLORES
15	105637	307	4.18	250	81.43	HANZO San Isidro	SAN ISIDRO

Download

20 rows

3.118s runtime



Resumo das análises realizadas

Pergunta	Métricas utilizadas	Técnica de análise	Resultado esperado
Restaurantes mais populares e bem avaliados	Nº de avaliações e pontuação média	JOIN + GROUP BY + AVG + COUNT	Identificação do perfil de popularidade e qualidade
Distritos com maior popularidade e qualidade	Nº de restaurantes, avaliações e média	GROUP BY + filtros	Comparação entre distritos
Restaurantes acima da média do distrito	Média do restaurante e média do distrito	JOIN + cálculo de diferença	Identificação de desempenho relativo

Pergunta	Métricas utilizadas	Técnica de análise	Resultado esperado
Emoções das opiniões	Sentimento e emoção predominante	GROUP BY + COUNT	Distribuição das emoções
Popularidade × satisfação	Nº de avaliações e % positivas	Agregação + cálculo percentual	Identificação de restaurantes com alta popularidade e satisfação

Visualizações desenvolvidas

Para facilitar a interpretação dos resultados, foram utilizadas diferentes visualizações:

1. Gráfico de dispersão – Popularidade × Pontuação média;
2. Gráfico de barras – Concentração de restaurantes por distrito;
3. Gráfico de barras – Diferença da pontuação em relação à média do distrito;
4. Gráfico de barras – Emoções predominantes segundo o sentimento;
5. Gráfico de dispersão – Popularidade × Satisfação.

Essas visualizações complementam as tabelas resultantes das consultas e permitem interpretar os padrões encontrados nos dados. O notebook utiliza matplotlib e conversões para Pandas (toPandas()) para gerar os gráficos a partir dos resultados obtidos com PySpark.

Conclusão da análise

A etapa de análise permitiu transformar as tabelas processadas do projeto em indicadores de negócio relacionados à popularidade, qualidade percebida, distribuição geográfica, desempenho relativo, sentimento e satisfação dos clientes.

As cinco perguntas foram respondidas por meio de consultas e transformações sobre dim_restaurant e fact_reviews, utilizando critérios quantitativos para evitar conclusões baseadas em poucos registros.

O código completo da análise está disponível no repositório público do projeto:

[04_analysis_peruvian_food_reviews.ipynb – GitHub](#)

Dessa maneira, a etapa de análise constitui a camada final do pipeline, na qual os dados previamente ingeridos, tratados e modelados são utilizados para gerar informações e visualizações que respondem diretamente às perguntas de negócio definidas para o projeto.

7. Autoavaliação:

Ao finalizar o desenvolvimento deste projeto, considero que os objetivos definidos inicialmente foram alcançados de forma satisfatória. Foi desenvolvido um pipeline de dados utilizando Databricks, PySpark e Python, contemplando as etapas de ingestão, qualidade, transformação, modelagem e análise dos dados. Além disso, foram construídas tabelas estruturadas e realizadas análises para responder às questões propostas, acompanhadas de visualizações para facilitar a interpretação dos resultados.

Durante a execução, as principais dificuldades estiveram relacionadas ao processamento de um volume significativo de dados, à identificação e tratamento de registros inválidos, valores nulos e informações indisponíveis, além da organização do projeto em diferentes etapas do pipeline. Também foi necessário integrar recursos de PySpark e Matplotlib para realizar o processamento e a apresentação dos resultados de forma adequada.

Como trabalhos futuros, pretende-se automatizar a execução do pipeline por meio de workflows do Databricks, implementar mecanismos mais avançados de monitoramento da qualidade dos dados e incorporar novas fontes de informação. Também seria possível desenvolver um dashboard interativo e atualizar periodicamente os indicadores, tornando a solução mais completa, reutilizável e adequada para utilização como projeto de portfólio.